Reviewer Pack — Coxeter Group Action Spectrum and AC⁰ Parity Circuit Depth

Entry c708ecff77fa · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 11:07:37 UTC

Coxeter Group Action Spectrum and AC⁰ Parity Circuit Depth

Entry ID: `c708ecff77fa` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 11:07:37 UTC

1. Conjecture

Field A (mathematical branch): Coxeter Group Theory **Field B** (complexity object): Complexity Theory: AC⁰ Parity Circuit Complexity

Statement:

> ['For every AC⁰ parity circuit C with size n, there exists a Coxeter group action on the set of variables of C such that the spectral radius of the induced representation on the vector space of circuit outputs is at least σ(C) / log(n), where σ(C) is the depth of C.', 'For all circuits computing PARITY with depth d and size n, there exists a Coxeter group action on the set of variables with spectral radius at least (1/4)d / log(n).', 'If a circuit C computes PARITY with depth d and no Coxeter group action can induce a representation on its outputs with spectral radius greater than (1/4)d / log(n), then the size of C is at most $2^{(n}$c/2 + dn^{d/4}) for some constant c.']

Rationale (proposer's reasoning):

> ['Coxeter group actions provide a rich source of combinatorial structures that can be used to study representation theory and its applications in complexity theory. The spectral radius of a representation can capture the complexity of computations, especially those involving parity functions.', 'AC⁰ parity circuits are known to be a challenging class for lower bound proofs. A connection with Coxeter group actions could reveal new insights into the structure of these circuits and potentially lead to improved lower bounds.', 'Previous work on algebraic approaches to circuit complexity has shown that certain invariants, such as the rank of matrix representations, can provide meaningful bounds on circuit size. The use of Coxeter group action spectra as a potential invariant for AC⁰ parity circuits is a novel idea with promising implications.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `83a033e78b9a2b63`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The spectral radius of the induced representation on the vector space of circuit outputs must be at least σ(C) / log(n) for AC^0 parity circuits.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def spectral_radius(matrix):
        n = len(matrix)
        identity = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
        eigenvalues = power_method(matrix, identity, n, max_iter=1000)
        return max(abs(e) for e in eigenvalues)

    def power_method(A, v0, n, max_iter):
        v = v0
        for _ in range(max_iter):
            v = matrix_multiply(A, v)
            norm = sum(x**2 for x in v)**0.5
            v = [x / norm for x in v]
        return v

    def matrix_multiply(A, B):
        n = len(A)
        result = [[sum(A[i][k] * B[k][j] for k in range(n)) for j in range(n)] for i in range(n)]
        return result

    def generate_circuit(depth: int, size: int):
        if depth == 1:
            return [random.choice([0, 1])]
        else:
            sub_depth = random.randint(1, depth - 1)
```

```
            sub_size = random.randint(1, size // 2)
            left = generate_circuit(sub_depth, sub_size)
            right = generate_circuit(depth - sub_depth, size - sub_size)
            return [random.choice([0, 1]) if i == 0 else (left[i] + right[i]) % 2 for i in range(s

    def is_ac0_parity(circuit):
        return all(x in {0, 1} for x in circuit)

    def depth_of_circuit(circuit):
        if len(circuit) == 1:
            return 1
        else:
            left_depth = depth_of_circuit(circuit[:len(circuit)//2])
            right_depth = depth_of_circuit(circuit[len(circuit)//2:])
            return max(left_depth, right_depth) + 1

    def size_of_circuit(circuit):
        return len(circuit)

    n = random.randint(5, 40)
    circuit = generate_circuit(n, n)

    if not is_ac0_parity(circuit):
        return {
            "metric_name": "spectral_radius",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "not_ac0_parity"
        }

    depth = depth_of_circuit(circuit)
    size = size_of_circuit(circuit)

    sigma_C = depth
    spectral_rad = spectral_radius([[circuit[i]] for i in range(size)])

    conjecture_holds = spectral_rad >= sigma_C / math.log(n)
    counterexample = "" if conjecture_holds else f"depth={depth}, size={size}, sigma_C={sigma_C},

    return {
        "metric_name": "spectral_radius",
        "metric_value": spectral_rad,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)
```

```
    mean = sum(r['metric_value'] for r in results) / len(results)
    std_dev = (sum((r['metric_value'] - mean)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"{results[next(i for i, r in enumerate(results)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b3e3a31.py", line 98, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b3e3a31.py", line 65, in run_trial
    circuit = generate_circuit(n, n)
              ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b3e3a31.py", line 46, in generate_circ
    left = generate_circuit(sub_depth, sub_size)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b3e3a31.py", line 46, in generate_circ
    left = generate_circuit(sub_depth, sub_size)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b3e3a31.py", line 46, in generate_circ
    left = generate_circuit(sub_depth, sub_size)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b3e3a31.py", line 45, in generate_circ
    sub_size = random.randint(1, size // 2)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 319, in randrange
    raise ValueError(f"empty range in randrange({start}, {stop})")
ValueError: empty range in randrange(1, 1)
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13689
2	preregistration	ollama_remote	glm4:latest	0	0	5949
3	novelty	ollama_remote	glm4:latest	0	0	4724
4	novelty	ollama_remote	glm4:latest	0	0	4994
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44175
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12800
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10161
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12489
9	judge	ollama_remote	glm4:latest	0	0	11767

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 120747 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c708ecff77fa.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c708ecff77fa.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c708ecff77fa.tar.gz` (if generated)